Andromeda Spec 03: Decode

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `decode_stage`, `decoder` · **Milestone:** Instruction dispatch, issue and register renaming

1. What this unit does

Decode turns each 32-bit RISC-V instruction into a `uop_t`: a record of what the instruction does, which registers it reads and writes, its immediate, and which execution unit it needs. It handles 4 instructions per cycle. It also catches unpredicted JALs early and redirects fetch, and marks illegal instructions so they trap when they reach the ROB head.

2. How it works

2.1 One instruction, one uop

RISC-V instructions are simple enough that each maps to exactly one uop. Nothing is cracked into multiple operations: a store is one uop that needs both its base and data registers, and an FMA is one uop with three sources. The term "uop" is borrowed from out-of-order literature, where the in-flight record is usually called a micro-op even in RISC cores.

Decode fills only the decode fields of the uop. Rename adds physical registers and dispatch adds queue indices later (file 00 lists all fields).

2.2 What the decoder extracts

- **Operation and unit:** `fu_type` picks the port, `op` the operation within the unit.
- **Registers:** which of rs1, rs2, rs3 are used (`rsN_en`), whether each is an integer or FP register, and the same for rd. Reads of x0 are treated as "not used"; writes to x0 set `rd_en = 0`, so rename allocates nothing for them.
- **Immediate:** sign-extended to 32 bits from whichever of the I, S, B, U or J formats applies, with `use_imm` set so the ALU takes it instead of rs2.
- **FP and memory details:** rounding mode, access size and signedness.
- **Exceptions:** illegal encodings set `exc_valid` with cause 2, and slots with `fetch_err` get cause 1. ECALL and EBREAK are marked as serializing CSR-type uops, and their exceptions (causes 11 and 3) are raised by the ROB at commit.

2.3 Routing of instruction classes

Instructions	fu_type	Port
LUI, AUIPC, OP, OP-IMM	ALU	P0 or P1
BEQ to BGEU, JAL, JALR	BRU	P1
CSRRW/S/C and immediate forms, ECALL, EBREAK, MRET, WFI, FENCE, FENCE.I	CSR (serializing)	P1

Instructions	fu_type	Port
F arithmetic, FMA, FSGNJ, FMIN/FMAX, compares, FCLASS, FCVT, FMV	FPU	P2
LB to LW, LBU, LHU, SB, SH, SW, FLW, FSW	LSU	P3

2.4 Early JAL redirect

A JAL's target (PC + imm) is fully known from its encoding. If the predictor didn't predict it taken, decode doesn't wait for execute: it raises `dec_redirect_valid` with the target, invalidates the later slots in its group, and fetch restarts. This saves several cycles on every BTB miss for a JAL. JALR and conditional branches can't be resolved here because they depend on register values.

3. Interface

3.1 decode_stage

Signal	Dir	Width	Description
`flush`	in	1	`be_flush` or `fe_flush`: drop the held group
`in_valid` / `in_slot`	in	4 / 4 × fetch_slot_t	From fetch_buffer
`in_ready`	out	1	Group accepted
`out_valid`	out	4	Valid per uop
`out_uop`	out	4 × uop_t	Decoded uops (decode fields filled)
`out_ready`	in	1	Rename took the group
`dec_redirect_valid` / `dec_redirect_pc`	out	1 / 32	Unpredicted JAL: refetch from its target; later slots in the group are invalidated

3.2 decoder (one per slot, combinational)

Signal	Dir	Width	Description
`slot`	in	fetch_slot_t	Instruction, PC and prediction
`uop`	out	uop_t	Decoded fields
`is_jal` / `jal_target`	out	1 / 32	Used by decode_stage for the redirect

4. Implementation notes

- **Generate the decode table.** The official riscv-opcodes repository lists every instruction's match and mask bits. A small script that turns it into a SystemVerilog `casez` or a set of match/mask comparisons avoids transcription errors and makes adding M later a one-line change.

- **Illegal detection is a whitelist.** Anything that doesn't match a supported encoding is illegal, including reserved funct fields and, for FP, reserved rounding modes in the encoding itself. A whitelist is easier to get right than trying to list illegal cases.
- **Keep it shallow.** Decode is one combinational stage between two registers. It is mostly wide AND/OR logic and should be fast, but watch the immediate muxing and the redirect path, which goes back into fetch.
- **Only the first unpredicted JAL redirects.** If a group has two, the first one redirects and the second is invalidated with the rest of the group.
- **Hold correctly.** When rename isn't ready, `decode_stage` keeps its registered group and drops `in_ready`. Make sure a flush in the same cycle still clears the held group.

5. Verification checklist

- ☐ Every RV32I, F and Zicsr instruction decodes to the expected fields (compare against a reference decoder in the testbench)
- ☐ Random 32-bit words: everything not in the whitelist is flagged illegal
- ☐ Writes to x0 never request a destination register
- ☐ Unpredicted JAL in each of the 4 slots redirects correctly and invalidates later slots
- ☐ Two unpredicted JALs in one group: only the first redirects
- ☐ Stall from rename holds the group without loss; flush during a stall clears it